

CLAUDE.md versus hooks for formatting and company rules

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN Session 6 of 12 · English notes

Session 6 (13 August 2026) made the contrast that Foundation answers keep mixing up: **CLAUDE.md** (and skills) tell the *model* how to write code; **hooks** run *locally* in the Claude Code lifecycle and do not consume generation tokens. The live example was Prettier via a post-tool-use hook after an Edit/Write, versus putting “format as Prettier / company style” in memory so the LLM rewrites the file in a second billed pass.

HOOK EVENT NAMES IN DOCS MAY DIFFER

The trainer listed pre-tool use, post-tool use, user prompt submit, session start/end, stop, sub-agent stop, notification, and pre-compact. Confirm the live event catalogue in [Claude Code docs](#). Quote escaping in `settings.json` and whether the IDE chat versus terminal actually fires hooks also drifted during the demo (a preview “Claude hook” setting had to be enabled).

Two ways to get “company format”

Approach	Where it lives	Who runs it	Tokens	Best for
CLAUDE.md / skills	Markdown in the repo (always-on or slash)	The model, as extra instructions every request	Yes — format rules are re-read and often re-applied as generation	Architecture, stack choice, SRP, “use Mermaid,” domain rules the model must understand
Hook (command in settings.json)	Claude Code settings + scripts in the project	Sandbox/local shell after or before a tool	No — NPX Prettier on disk is not an LLM turn	Mechanical format, lint --fix, block rm, protect .env

The trainer’s “ideal situation”: do *not* ask the LLM to pretty-print. Use hooks as an extended plugin on the workspace. Lalit’s question — “hooks won’t be counted as token usage, right?” — answer: yes, because NPX (or any shell) runs on the local system / sandbox, not as model output.

What a post-tool-use formatter actually did

Prompt: add a hello message on the frontend login page. Claude invoked Edit/Write (IO tools). After the write, the **post-tool-use** hook ran: `echo status` (visible in the terminal-based Claude session — the GUI had no dedicated hook log) and `npx prettier` on the touched file. Diff: single quotes to double quotes, long JSX broken across lines — clean-code mechanics, not new product behaviour. Prettier options (semicolons, trailing commas, indent) belong in Prettier config consumed by the hook, not in a paragraph of CLAUDE.md.

- Hook bodies are Linux/macOS commands (and whatever the sandbox shell can run). Double quotes in JSON must be escaped; bad quoting was why the previous day’s hook never fired.
- Status strings you embed show up in tool execution UI; they are not a special JSON protocol unless you choose to emit JSON.

- Verify hooks from **terminal** Claude if the VS Code chat surface is silent — classmates saw terminal-only firing until a preview setting was flipped.

Keep CLAUDE.md for things Prettier cannot know

Memory still wins for: production-ready defaults, which framework, “controllers before services,” Mermaid-not-JPEG, OpenAPI path, never invent fields. Those require the model to *choose* differently. Formatting, secret-file exclusion, and “never run this bash” are better as hooks because they are deterministic and cheap. Putting both in CLAUDE.md causes the model to narrate format passes you could have got for free.

Lifecycle: hooks are events, not skills

Skills still inject roles when slashed. Hooks fire on Claude’s activity stream regardless of which skill was used. Session start/end, prompt submit, compact, sub-agent stop — each can run a command. That is how you “manage the lifecycle of Claude’s interaction with LLMs” in the trainer’s phrase. You configure them in `settings.json` with matchers (which tool: Edit, Write, Bash, ...).

CLAUDE.MD CANNOT STOP AUTO-MODE BASH

In auto/bypass permission mode the model will not ask before `rm`. A sentence in CLAUDE.md (“never delete files”) is a suggestion. A **pre-tool-use** matcher that runs `block-dangerous-commands.sh` can deny the tool. That pairing is the next chapter; this chapter’s exam line is: format and policy enforcement that must always happen belong in hooks, not in memory prose.

Study check

Written scenario: “team style guide is 2-space indent and double quotes; they also must use repository pattern.” Split the answer: indent/quotes → post-tool Prettier hook; repository pattern → backend CLAUDE.md or a backend skill. Mixing them into one CLAUDE.md blob is the distractor.

Official documentation

- **Claude Code overview**

<https://code.claude.com/docs/en/overview>

Hooks, settings.json, and how they relate to CLAUDE.md.

- **Claude Docs home**

<https://docs.claude.com/en/home>

If hook event names were renamed after August 2026.

- **Guardrails**

<https://docs.claude.com/en/docs/test-and-evaluate/strengthen-guardrails>

Deterministic checks belong outside the model when possible.

- **Prompting overview**

<https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>

Why extra format instructions in the prompt are the expensive duplicate of a hook.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.